FederHub Platform User Stories

Overview

This document presents the unified user stories for the FederHub project, focusing on the key personas: Platform Admin, ML Engineer (Federation Owner), and Client Operator. The platform supports privacy-preserving machine learning through decentralized orchestration, secure data handling, and metric monitoring.

Structured under 5 epics, each story adheres to the INVEST framework, MOSCOW prioritization, T-shirt sizing, and the 3 C's for agile development.

Epic 1: Platform Management and Security

Focus: Organization onboarding, access control, and platform-wide security.

Story 1.1: As a Platform Admin, I want to manage organizations and users, so that platform access is controlled and secure.

- **Acceptance Criteria:**
  - Given an admin is logged in, when adding a new organization, then a unique API key/Auth Token is generated.
  - Given a deactivated user, when they attempt to log in, then JWT authentication instantly fails and access is denied.

- **Priority:** Must
- **Sizing:** M
- **3 C's: Card:** This story card. **Conversation:** Discuss security tiers and JWT lifecycle. **Confirmation:** Verify via organization creation and test user deactivation.
- **INVEST:** Independent (standalone org management), Negotiable (auth methods flexible), Valuable (security), Estimable (known API endpoints), Small (fits sprint), Testable (login/access tests).

Epic 2: Job Definition and Experimentation

Focus: Hyperparameter configuration, model templates, and training orchestration.

Story 2.1: As an ML Engineer, I want to create federated training jobs, so that the orchestration engine can manage the training cycle.

- **Acceptance Criteria:**

    - Given the job setup interface, when parameters (rounds, local epochs) are input, then the job is saved to the pending queue.
    - Given missing parameters, when submitting, then inline validation errors prompt for completion.

- **Priority:** Must
- **Sizing:** M
- **3 C's: Card:** This story. **Conversation:** Define necessary ML parameters for FedAvg. **Confirmation:** Generate and validate a queued job.
- **INVEST:** Independent, Negotiable, Valuable, Estimable, Small, Testable.

Story 2.2: As an ML Engineer, I want to upload baseline model templates, so that the server has a starting point to distribute to clients.

- **Acceptance Criteria:**
    - Given a PyTorch model file, when uploaded to the registry, then it is tagged with a unique version ID.
    - Given an active job, when a model is selected, then it is successfully linked to that training cycle.
- **Priority:** Should
- **Sizing:** S
- **3 C's: Card:** This story. **Conversation:** Categorize model formats (.pt, .h5). **Confirmation:** Test file upload and versioning mapping.
- **INVEST:** Independent, Negotiable, Valuable, Estimable, Small, Testable.

Epic 3: Edge Execution and Data Management

Focus: Secure local agent setup, data mounting, and local training loops.

Story 3.1: As a Client Operator, I want to select my local data directory via the UI, so that the agent can read files without uploading them.

- **Acceptance Criteria:**
    - Given the local desktop app, when browsing for data, then the native OS file explorer captures the local path.
    - Given a selected path, when the training starts, then the directory is dynamically mounted as a read-only Docker volume.
- **Priority:** Must
- **Sizing:** M
- **3 C's: Card:** This story. **Conversation:** Align with Docker volume security policies. **Confirmation:** Test volume mounts and verify zero outbound file transfers.

- **INVEST:** Independent, Negotiable, Valuable (data privacy), Estimable, Small, Testable.

Story 3.2: As a Client Operator, I want to authenticate my local agent, so that it can securely connect to the central aggregator.

- **Acceptance Criteria:**
  - Given valid credentials, when submitted in the local app, then a JWT is successfully received from the FastAPI backend.
  - Given a successful connection, when waiting for a job, then the UI status updates to "Connected: Standby."
- **Priority:** Must
- **Sizing:** S
- **3 C's: Card:** This story. **Conversation:** Review token storage in local agent. **Confirmation:** Verify API handshake and token receipt.
- **INVEST:** Independent, Negotiable, Valuable, Estimable, Small, Testable.

Epic 4: Monitoring and Metrics

Focus: Dashboards, global accuracy tracking, and local telemetry.

Story 4.1: As an ML Engineer, I want to view a real-time global metrics dashboard, so that I can track AI model improvements across rounds.

- **Acceptance Criteria:**
  - Given a completed aggregation round, when viewing the dashboard, then a line chart updates automatically with the new Global Accuracy.
  - Given a failed client node, when detected, then an alert is shown on the dashboard.
- **Priority:** Must
- **Sizing:** M
- **3 C's: Card:** This story. **Conversation:** Integrate gRPC metrics sources to React. **Confirmation:** View dashboards and test round updates.
- **INVEST:** Independent, Negotiable, Valuable, Estimable, Small, Testable.

Story 4.2: As a Client Operator, I want to view my local node's training metrics, so that I can understand my contribution to the global model.

- **Acceptance Criteria:**
  - Given an active local training run, when in progress, then the UI parses standard output (stdout) to show local loss/accuracy.

    - Given a completed round, when summarized, then local accuracy is displayed alongside the server-provided global average.
- **Priority:** Should
- **Sizing:** S
- **3 C's: Card:** This story. **Conversation:** Format local metrics for business readability. **Confirmation:** Generate local mock training and review summaries.
- **INVEST:** Independent, Negotiable, Valuable, Estimable, Small, Testable.

Epic 5: Non-User Stories & Technical Foundations

Focus: Core infrastructure constraints, communication protocols, and strict data compliance.

Story 5.1: As a Developer, I need to implement a bi-directional gRPC communication channel, so that massive arrays of model weights transmit securely.

- **Acceptance Criteria:**
    - Given a PyTorch model, when sent by the server, then it is serialized into Protobuf format.
    - Given the client container, when receiving the payload, then it successfully deserializes the weights into a local model.
- **Priority:** Must
- **Sizing:** L
- **3 C's: Card:** This story. **Conversation:** Map gRPC logic to Docker network endpoints. **Confirmation:** Simulate weight transfers and check integrity.
- **INVEST:** Independent, Negotiable, Valuable, Estimable, Small, Testable.

Story 5.2: As a Stakeholder, I want strict enforcement preventing outbound raw data transfer, so that the platform complies with HIPAA/GDPR. *(Constraint)*

- **Acceptance Criteria:**
    - Given the client application, when executing a training job, then network monitoring confirms no HTTP POST requests containing images or CSVs occur.
    - Given a telemetry capture, when analyzed, then only mathematical tensors (weights/gradients) are transmitted.
- **Priority:** Must
- **Sizing:** M
- **3 C's: Card:** This story. **Conversation:** Define exact network governance policies. **Confirmation:** Enforce isolation and run packet capture tests.
- **INVEST:** Independent, Negotiable, Valuable, Estimable, Small, Testable.